ĐẠI HỌC ĐÀ NẴNG

TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN TỬ - VIỄN THÔNG

BÁO CÁO HỌC PHẦN

Nhóm: 12

SO SÁNH HIỆU QUẢ MÔ HÌNH CNN TỰ XÂY DỰNG VÀ REGNETY-3.2GF TRONG PHÂN LOẠI ẢNH KHÓI VÀ LỬA

Nhóm sinh viên thực hiện: Nguyễn Đức Trúc

Nguyễn Tấn Vỹ

Nguyễn Trường Thọ

Người hướng dẫn: Trần Thị Minh Hạnh

Đà Nẵng, tháng 10/2026

PHÂN LOẠI HÌNH ẢNH KHÓI VÀ LỬA BẰNG CNN TỰ XÂY DỰNG VỚI CONV1 7×7 VÀ REGNETY-3.2GF

Tóm tắt

Đề tài nghiên cứu bài toán phân loại hình ảnh khói và lửa thành bốn nhãn: không có khói/lửa (none), chỉ có khói (smoke), chỉ có lửa (fire) và đồng thời có khói với lửa (smoke_fire). Tập dữ liệu thực nghiệm gồm 21.527 ảnh từ D-Fire, được phân tách thành 13.779 ảnh huấn luyện, 3.442 ảnh xác thực và 4.306 ảnh kiểm tra. Hai hướng tiếp cận được khảo sát: CNN tự xây dựng có lớp tích chập đầu tiên sử dụng bộ lọc 7×7 (CNN-K7) và RegNetY-3.2GF dùng trọng số tiền huấn luyện trên ImageNet-1K rồi tinh chỉnh cho bốn lớp đích.

Kết quả trên tập kiểm tra cho thấy CNN-K7 đạt Accuracy 82,23% và Macro F1 73,75%, trong khi RegNetY-3.2GF đạt Accuracy 93,59% và Macro F1 90,22%. CNN-K7 có 426.628 tham số, nhỏ hơn đáng kể so với 17.929.390 tham số của RegNetY-3.2GF. Hạn chế nổi bật của CNN-K7 là khả năng phân biệt nhãn fire với smoke_fire: trong 220 ảnh fire, 128 ảnh bị dự đoán thành smoke_fire. Báo cáo trình bày kiến trúc, cấu hình, đường cong học, ma trận nhầm lẫn và phân tích sự đánh đổi giữa độ chính xác phân loại bốn lớp và độ phức tạp mô hình.

Phạm vi nghiên cứu

Báo cáo tập trung vào nhiệm vụ phân loại toàn ảnh; đầu ra là một trong bốn lớp. Hệ thống chưa xác định tọa độ vùng cháy, chưa xử lý video thời gian thực và chưa được kiểm chứng trong các tình huống triển khai thực địa. Năm mục chính lần lượt trình bày bối cảnh, phương pháp liên quan, quy trình triển khai, kết quả thực nghiệm và kết luận. Các đoạn mã tiêu biểu và biểu mẫu dành cho báo cáo nhóm được đặt ở phần phụ lục.

1. Giới thiệu

1.1. Bối cảnh và vấn đề nghiên cứu

Cháy và khói có thể xuất hiện dưới nhiều điều kiện quan sát: ban ngày, ban đêm, nền cảnh phức tạp, ánh sáng mạnh hoặc đốm lửa có kích thước nhỏ. Phương pháp dựa trên ngưỡng màu thường nhạy với vật thể có màu và cường độ sáng tương tự ngọn lửa; ngược lại, khói có thể khó tách khỏi mây, sương hoặc nền ít tương phản. Các mạng nơ-ron tích chập được sử dụng để học đặc trưng từ ảnh thay vì chỉ áp dụng một tập quy tắc thủ công [1], [3].

Đối với một hệ thống giám sát hình ảnh, câu hỏi không dừng ở việc có dự đoán đúng đa số ảnh hay không. Tỷ lệ bỏ sót những trường hợp thật sự có lửa, mức nhầm lẫn giữa các lớp và chi phí tính toán của mô hình cũng cần được đánh giá. Vì lớp none chiếm gần một nửa tập kiểm tra, Accuracy có thể che khuất mức độ yếu ở lớp fire nếu chỉ sử dụng duy nhất một chỉ số tổng hợp.

1.2. Mục tiêu và yêu cầu kỹ thuật

Mục tiêu đặt ra là xây dựng một bộ phân loại ảnh RGB kích thước 224×224 cho bốn trạng thái nêu trên; triển khai CNN tự xây dựng với lớp Conv1 dùng kernel 7×7; đồng thời khảo sát một mạng hiện đại có sẵn là RegNetY-3.2GF để hình thành cơ sở so sánh. Phần đánh giá sử dụng Accuracy, Macro Precision, Macro Recall, Macro F1 và ma trận nhầm lẫn trên cùng tập kiểm tra.

Giới hạn kỹ thuật của hướng CNN-K7 được thể hiện rõ trong mã nguồn: bốn khối tích chập, trong đó Conv1 có kernel 7×7 và Conv2–Conv4 dùng kernel 3×3. Kiến trúc này không đồng nghĩa với việc toàn bộ lớp tích chập đều có kernel 7×7. Thiết kế khớp với yêu cầu lựa chọn một trong các kích thước 7×7, 9×9 hoặc 11×11 cho lớp tích chập đầu tiên.

1.3. Quy trình và đóng góp

Hình 1 tóm tắt luồng thực nghiệm từ dữ liệu gốc đến quá trình đánh giá. Cùng tệp manifest được dùng cho hai kiến trúc nhằm giữ cố định thành phần train/validation/test; việc huấn luyện và lưu checkpoint của từng mô hình được thực hiện độc lập. Các kết quả đầu ra gồm lịch sử huấn luyện, trọng số tốt nhất được chọn theo validation loss, ma trận nhầm lẫn và hình trực quan hóa đặc trưng.



Hình 1. Quy trình thực nghiệm phân loại bốn lớp trên tập ảnh D-Fire (sơ đồ xây dựng từ mã nguồn dự án).

2. Các nghiên cứu/phương pháp liên quan

2.1. Phương pháp truyền thống và học sâu

Các tiếp cận học máy truyền thống thường thiết kế thủ công đặc trưng như màu sắc, histogram, kết cấu hoặc biên rồi sử dụng thuật toán phân lớp. Cách này có ưu điểm đơn giản và dễ phân tích nhưng thường gặp khó khăn khi nền cảnh, ánh sáng và hình dạng khói/lửa biến đổi nhiều. CNN sử dụng các lớp tích chập nối tiếp để hình thành đặc trưng từ thấp đến cao, đồng thời có thể tối ưu đồng thời bộ lọc và lớp phân loại [3].

Bảng 1 so sánh đặc trưng của hai hướng tiếp cận đối với bài toán trong phạm vi báo cáo. Các nhận xét ở bảng phản ánh đặc tính phương pháp, không phải các chỉ số thực nghiệm riêng của dự án.

Bảng 1. Ưu điểm và hạn chế của các nhóm phương pháp phân loại ảnh.

Phương pháp	Ưu điểm	Hạn chế

Phương pháp	Ưu điểm	Hạn chế
Đặc trưng thủ công + ML	Mô hình tương đối nhỏ; dễ diễn giải; ít chi phí huấn luyện.	Phụ thuộc đặc trưng thiết kế; nhạy với nền cảnh/ánh sáng.
CNN tự xây dựng	Học đặc trưng tự động; cấu hình linh hoạt; có thể tối ưu kích thước.	Nhạy với kiến trúc, tập dữ liệu và siêu tham số; cần huấn luyện đủ.
Học chuyển giao với CNN	Tận dụng đặc trưng học trước; hiệu quả khi bài toán gần dữ liệu ảnh tự nhiên.	Mô hình nhiều tham số; đòi hỏi RAM/VRAM và kiểm soát quá khớp.

2.2. Phép tích chập, kernel và đặc trưng đa tầng

Tại một vị trí không gian, phép tích chập lấy tích vô hướng giữa cửa sổ ảnh và bộ lọc học được, cộng hệ số dịch rồi tạo ra giá trị trên feature map. Kernel 7×7 bao phủ lân cận rộng hơn kernel 3×3 trong một lớp riêng lẻ, nhưng yêu cầu nhiều phép nhân hơn. Khi dùng padding=3 và stride=1, phép tích chập 7×7 có thể giữ nguyên chiều cao, chiều rộng của ảnh đầu vào; thao tác MaxPool2d(2) sau đó giảm kích thước xuống một nửa.

Với CNN trong PyTorch, phép toán Conv2d được triển khai theo dạng tương quan chéo (cross-correlation): các trọng số được nhân với giá trị của vùng ảnh tương ứng, cộng trên toàn bộ kênh đầu vào và bổ sung độ lệch. Biểu thức tính một phần tử đầu ra tại kênh o được viết như sau:

$$Z_{o,i,j} = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_{u=0}^{K-1} \sum_{v=0}^{K-1} W_{o,c,u,v} \widetilde{X}_{c,\, Si+u,\, Sj+v}$$

Trong biểu thức tích chập, X là tensor đầu vào sau khi đệm biên, W là trọng số kernel, b là bias, K là kích thước kernel, S là stride, c là chỉ số kênh đầu vào và o là chỉ số kênh đầu ra. Số kênh đầu ra bằng số bộ lọc.

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P - K}{S} \right\rfloor + 1$$

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2P - K}{S} \right\rfloor + 1$$

Áp dụng cho Conv1 có ảnh vào 224×224, kernel K=7, padding P=3 và stride S=1:

$$H_{\text{Conv1}} = \frac{224 + 2(3) - 7}{1} + 1 = 224$$

$$H_{\text{Pool1}} = \frac{224}{2} = 112$$

Do đó Conv1 tạo 32 bản đồ đặc trưng 224×224; phép MaxPool2d(2) mới làm giảm chiều không gian xuống 112×112. Bộ lọc 7×7 là kích thước cửa sổ tính tích chập, không phải kích thước feature map.

Batch Normalization giúp chuẩn hóa kích hoạt theo mini-batch trong huấn luyện [4]; hàm ReLU tạo phi tuyến; MaxPooling giảm độ phân giải không gian. Sau các khối trích xuất, Global Average Pooling (GAP) nén mỗi kênh đặc trưng thành một giá trị trước khi đưa tới lớp đầy đủ kết nối. Ở CNN-K7, GAP chuyển tensor 256×28×28 thành vectơ 256 phần tử, hỗ trợ giảm số tham số so với việc làm phẳng toàn bộ feature map.

Các lớp Batch Normalization, ReLU và Global Average Pooling (GAP) được mô tả lần lượt bằng các biểu thức sau:

$$\mathrm{BN}(x) = \gamma \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}} + \beta$$

$$\mathrm{ReLU}(x) = \max(0, x)$$

$$g_c = \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} A_{c,i,j}$$

Trong Batch Normalization, giá trị trung bình và phương sai được tính theo minibatch; γ và β là hai tham số học được, còn ε bảo đảm ổn định số học. Sau ReLU, các kích hoạt âm trở thành 0. Trong GAP, H và W lần lượt là chiều cao và chiều rộng của bản đồ đặc trưng; mỗi kênh được rút gọn về một giá trị trung bình.

Việc tăng kernel không bảo đảm chất lượng phân loại tăng theo. Một lớp 7×7 có trường tiếp nhận lớn hơn một lớp 3×3, nhưng năng lực biểu diễn còn phụ thuộc số kênh, độ sâu mạng, mức cân bằng dữ liệu và cách học. Do đó, hiệu quả của kernel được đánh giá bằng dữ liệu kiểm tra thực tế, không suy ra từ kích thước kernel đơn lẻ.

2.3. Học chuyển giao và kiến trúc RegNetY

Học chuyển giao sử dụng trọng số học từ nhiệm vụ nguồn làm điểm khởi tạo cho nhiệm vụ đích. Với ảnh tự nhiên, trọng số tiền huấn luyện từ ImageNet-1K thường mang theo các bộ lọc nhận diện biên, kết cấu và cấu trúc hình ảnh ở nhiều mức độ [2], [8]. Quá trình tinh chỉnh cho bốn lớp thay lớp phân loại cuối cùng và tối ưu lại một phần hoặc toàn bộ mạng.

RegNet là họ kiến trúc CNN được xây dựng theo hướng thiết kế không gian mạng có quy luật. Biến thể RegNetY kết hợp các khối bottleneck, tích chập theo nhóm (grouped convolution) và cơ chế Squeeze-and-Excitation nhằm hiệu chỉnh trọng số theo kênh [2], [6]. Thực nghiệm lựa chọn RegNetY-3.2GF với trọng số khởi tạo IMAGENET1K_V2 từ TorchVision [8], sau đó thay lớp phân loại cuối để dự đoán bốn nhãn.

Hình 4 tại Mục 3.4 diễn giải các thành phần chính của RegNetY-3.2GF gồm stem, bốn stage, Global Average Pooling và lớp phân loại. Trong mỗi stage là nhiều khối bottleneck có cơ chế tích chập nhóm và chú ý kênh; các stage trong sơ đồ không được hiểu là các lớp tích chập đơn lẻ.

2.4. Các chỉ số đánh giá

Ma trận nhầm lẫn được ký hiệu là M; phần tử ở hàng i, cột j cho biết số ảnh có nhãn thực tế là lớp i nhưng được dự đoán thành lớp j. Accuracy được tính bằng tổng các phần tử trên đường chéo chia cho số ảnh kiểm tra. Precision đo tỷ lệ dự đoán đúng trong các ảnh được gán vào một lớp, còn Recall đo tỷ lệ ảnh thực tế của lớp đó được nhận diện đúng.

Các công thức dưới đây xác định Precision và Recall cho lớp c theo một-với-phần-còn-lại (one-versus-rest).

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}$$

$$\text{Recall}_c = \frac{TP_c}{TP_c + FN_c}$$

F1-score là trung bình điều hòa của Precision và Recall. Trong biểu thức dưới đây, P_c là Precision của lớp c và R_c là Recall của lớp c; Macro F1 lấy trung bình F1-score của bốn lớp với trọng số bằng nhau.

$$F1_c = \frac{2P_cR_c}{P_c + R_c}, \qquad \text{MacroF1} = \frac{1}{C}\sum_{c=1}^{C} F1_c$$

$$\text{Accuracy} = \frac{1}{N}\sum_{c=1}^{C} M_{cc}, \qquad \text{MacroRecall} = \frac{1}{C}\sum_{c=1}^{C} R_c$$

Ký hiệu M là ma trận nhầm lẫn; phần tử nằm trên đường chéo chính ứng với lớp c là số ảnh lớp c được dự đoán đúng. Trong công thức Macro Recall, R_c là Recall của lớp c. N là tổng số ảnh kiểm tra và C = 4 là số lớp. Khi dữ liệu mất cân bằng, Macro F1 đặc biệt hữu ích vì lớp fire có ít ảnh hơn lớp none.

Các giá trị Macro là trung bình không trọng số của bốn lớp, vì vậy mỗi lớp có ảnh hưởng ngang nhau bất kể số ảnh. Đây là chỉ số quan trọng khi dữ liệu bị lệch lớp. Bên cạnh đánh giá bốn lớp, có thể thiết lập mục tiêu nhị phân "có lửa" bằng cách gộp fire và smoke_fire; phép gộp này mang ý nghĩa cảnh báo nhưng không thay thế đánh giá phân loại bốn lớp.

2.5. Hàm mất mát và phương pháp tối ưu

Bộ phân loại xuất ra bốn giá trị logits. Trong lúc huấn luyện, CrossEntropyLoss nhận trực tiếp logits và nhãn nguyên, áp dụng log-softmax bên trong để tính mất mát. Softmax chỉ là bước chuyển logits thành xác suất khi cần diễn giải kết quả, không phải một lớp được gắn thêm trước CrossEntropyLoss:

$$\hat{p}_{n,k} = \frac{\exp(z_{n,k})}{\sum_{j=1}^{C}\exp(z_{n,j})}$$

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N}\sum_{n=1}^{N}\log \hat{p}_{n,y_n}$$

Trong các biểu thức, z là logit của từng lớp trước softmax; chỉ số n biểu diễn mẫu ảnh và k biểu diễn lớp dự đoán. Ký hiệu y là nhãn đúng, N là số mẫu và C = 4 là số lớp. Hàm mất mát phạt các dự đoán có xác suất thấp cho nhãn thực tế.

AdamW được sử dụng cho cả hai mô hình. Thuật toán kết hợp ước lượng mô-men bậc nhất, bậc hai của gradient và weight decay tách rời [7]. Dạng cập nhật khái quát với hệ số suy giảm λ được viết như sau:

$$\theta_{t+1} = (1 - \eta_t \lambda)\theta_t - \eta_t \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \varepsilon}$$

Trong công thức AdamW, η là learning rate, λ là hệ số weight decay, còn các đại lượng m và v có dấu mũ biểu thị ước lượng mô-men bậc nhất và bậc hai sau hiệu chỉnh bias; ε là hằng số ổn định số học. AdamW cập nhật trọng số dựa vào gradient và tách riêng thành phần suy giảm trọng số.

2.6. Điều kiện đối chiếu và giới hạn suy luận

Hai mô hình cùng sử dụng bốn nhãn và tập train/validation/test giống nhau. Tuy nhiên, RegNetY được khởi tạo bằng trọng số pretrained ImageNet-1K V2 và huấn luyện theo hai pha, trong khi CNN-K7 huấn luyện từ đầu; vì thế chênh lệch hiệu năng phản ánh đồng thời kiến trúc, quy mô tham số, chiến lược khởi tạo và lịch huấn luyện, không thể quy toàn bộ cho kích thước kernel hay tên kiến trúc.

Tập validation dùng chọn checkpoint có validation loss thấp nhất. Tập test chỉ dành cho báo cáo cuối cùng và phân tích lỗi, tránh sử dụng lặp lại để lựa chọn siêu tham số. Các giá trị thời gian ghi trong lịch sử huấn luyện phụ thuộc môi trường chạy, nên không dùng thay cho phép đo tốc độ suy luận được chuẩn hóa trên phần cứng chung.

3. Phương pháp thực hiện

3.1. Tập dữ liệu và phân chia

D-Fire là tập ảnh được công bố phục vụ nghiên cứu phát hiện khói và lửa [1], [9]. Bài toán gốc có thể đi kèm chú thích vị trí đối tượng; trong thực nghiệm này, mỗi ảnh được gán một nhãn ở mức toàn ảnh. Tệp class_to_idx.json quy định cố định bốn nhãn: none=0, smoke=1, fire=2, smoke_fire=3. Ba manifest ghi đường dẫn ảnh, nhãn số, tên lớp và tập chia tương ứng.

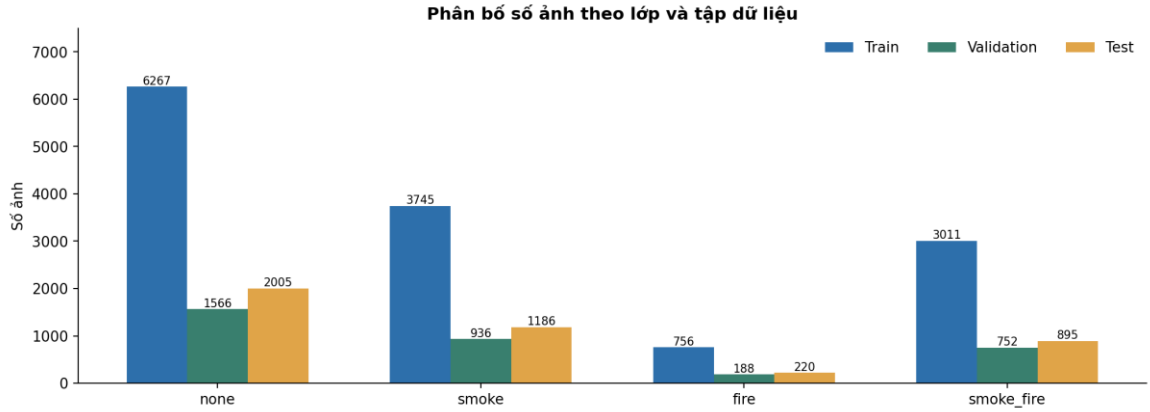
Bảng 2 thể hiện số ảnh của từng nhãn, trong đó lớp fire có quy mô nhỏ nhất. Các tệp train.csv, val.csv và test.csv trong dự án đối chứng trùng hoàn toàn với các tệp manifest của dự án CNN ban đầu (kiểm tra cả thứ tự dòng và nhãn), tạo cơ sở đối chiếu trực tiếp trên cùng ảnh kiểm tra.

Bảng 2. Phân bố số ảnh D-Fire theo lớp và theo tập (từ các tệp manifest).

Lớp	Train	Validation	Test	Tổng

Lớp	Train	Validation	Test	Tổng
none	6.267	1.566	2.005	9.838
smoke	3.745	936	1.186	5.867
fire	756	188	220	1.164
smoke_fire	3.011	752	895	4.658
Tổng	13.779	3.442	4.306	21.527

Hình 2 biểu diễn trực quan mức độ mất cân bằng giữa các lớp. Lớp none chiếm 2.005/4.306 ảnh kiểm tra, còn fire có 220 ảnh. Do đó, sự chênh lệch Accuracy ở mức vài phần trăm chưa phản ánh đầy đủ ảnh hưởng đối với lớp fire; kết quả theo lớp cần được trình bày cùng với chỉ số Macro.



Hình 2. Số lượng ảnh train, validation và test theo bốn nhãn (vẽ từ manifest của dự án).

3.2. Tiền xử lý và tăng cường dữ liệu

Mã nguồn data.py sử dụng ảnh đầu vào ở chế độ RGB và Resize trực tiếp về 224×224 pixel. Dữ liệu huấn luyện được áp dụng RandomHorizontalFlip với xác suất 0,5; xoay ngẫu nhiên trong khoảng ±10°; ColorJitter với brightness, contrast, saturation bằng 0,10 và hue bằng 0,02. Ảnh sau đó được chuyển thành tensor và chuẩn hóa theo mean=[0,485; 0,456; 0,406], std=[0,229; 0,224; 0,225]. Tập validation/test không dùng phép tăng cường ngẫu nhiên để đánh giá nhất quán.

Với từng kênh ảnh RGB, bước Normalize áp dụng phép biến đổi tuyến tính như sau:

$$x_c' = \frac{x_c - \mu_c}{\sigma_c}$$

$$\mu = (0.485, 0.456, 0.406), \quad \sigma = (0.229, 0.224, 0.225)$$

Ba cặp giá trị mean và std tương ứng các kênh R, G, B. Phép xoay, lật ngang và ColorJitter chỉ được áp dụng cho tập huấn luyện; tập validation và test dùng Resize, ToTensor, Normalize để bảo đảm phép đo có tính nhất quán. Việc Resize trực tiếp về 224×224 là cài đặt thực tế của dự án, không đồng nhất với toàn bộ pipeline suy luận mặc định được công bố cho trọng số ImageNet-1K V2 của TorchVision [8].

Dữ liệu đưa vào DataLoader theo batch_size=16, shuffle=True cho tập train, shuffle=False cho validation/test. Trong tệp mã được cung cấp, num_workers=0 và pin_memory=True. Phần tiền xử lý được sử dụng chung trong cả hai nhánh, làm giảm sai lệch do các quy tắc đọc ảnh khác nhau; tuy vậy, cách khởi tạo trọng số và lịch điều chỉnh learning rate của hai mô hình vẫn khác nhau.

3.3. Kiến trúc CNN tự xây dựng với Conv1 7×7

CNN-K7 nhận tensor đầu vào theo thứ tự kênh trước, kích thước 3×224×224. Block1 dùng Conv2d(3,32,kernel_size=7,padding=3), sau đó BatchNorm2d(32), ReLU và MaxPool2d(2). Ba khối sau dùng kernel 3×3, padding=1; trong đó Block2 và Block3 có MaxPool, Block4 không có MaxPool. Đầu ra sau Block4 có kích thước 256×28×28 và được đưa qua AdaptiveAvgPool2d(1).

Hình 3 là sơ đồ kiến trúc CNN-K7 dựng lại theo mã nguồn thực tế; mỗi khối tích chập được biểu diễn như một chồng feature map có chiều sâu tương ứng số kênh. Sơ đồ phân biệt rõ kích thước kernel và kích thước tensor sau từng khối, tránh nhầm lẫn giữa kernel 7×7 và feature map 7×7 như trong một số sơ đồ CNN tham khảo.

Số tham số có thể học được của lớp tích chập đầu tiên (bao gồm bias) được kiểm tra bằng công thức:

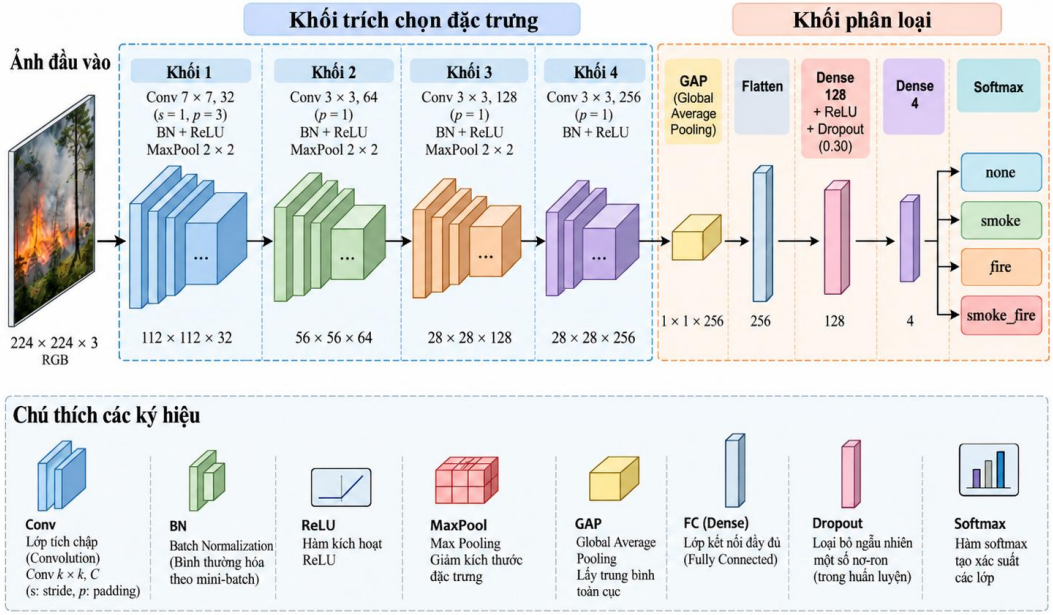
$$P_{\text{Conv1}} = 32\,(3 \times 7 \times 7 + 1) = 4\,736$$

Cộng tham số các lớp Conv2d, BatchNorm2d và Linear trong toàn mạng cho kết quả:

$$P_{\text{total}} = (4\,736 + 64) + (18\,496 + 128) + (73\,856 + 256)$$

$$+ (295\,168 + 512) + 32\,896 + 516 = 426\,628$$

Trong tổng trên, BatchNorm có hai tham số học được (γ và β) cho mỗi kênh; các thống kê running mean/running variance không được tính là tham số có gradient. Tổng 426.628 được xác nhận bằng hàm đếm tham số trong model_k7.py.



Hình 1. Kiến trúc mô hình CNN-K7 đề xuất.

Hình 3. Kiến trúc CNN-K7 với Conv1 7×7, ba lớp Conv 3×3 và đầu phân loại bốn lớp (theo model_k7.py).

Hai lớp tuyến tính cuối cùng gồm Linear(256,128), ReLU, Dropout(0,30) và Linear(128,4). Mô hình có 426.628 tham số có thể học, đã được xác nhận qua smoke test trên GPU với đầu vào [4,3,224,224] và đầu ra [4,4]. Cấu trúc được mô tả chi tiết ở Bảng 3, trong đó kích thước sau pooling được dùng làm đầu ra của từng khối.

Bảng 3. Thông số và kích thước tensor của các thành phần CNN-K7.

Thành phần	Cấu hình chính	Đầu ra
Input	RGB, 224×224	3×224×224
Block1	Conv 7×7 (3→32), BN, ReLU, Pool2	32×112×112
Block2	Conv 3×3 (32→64), BN, ReLU, Pool2	64×56×56
Block3	Conv 3×3 (64→128), BN, ReLU, Pool2	128×28×28
Block4	Conv 3×3 (128→256), BN, ReLU	256×28×28
GAP	AdaptiveAvgPool2d(1)	256
FC1	Linear 256→128, ReLU, Dropout 0,30	128
FC2	Linear 128→4	4 logits

3.4. Kiến trúc và học chuyển giao với RegNetY-3.2GF

Mô hình đối chứng được khởi tạo bằng torchvision.models.regnet_y_3_2gf với trọng số RegNet_Y_3_2GF_Weights.IMAGENET1K_V2. Lớp fully connected gốc được thay bằng Linear(1512,4), phù hợp số nhãn đầu ra. Sau khi thay lớp phân loại, số

tham số của mô hình là khoảng 17,929 triệu; kích thước này cao hơn CNN-K7 khoảng 42 lần, phản ánh độ phức tạp của backbone pretrained.

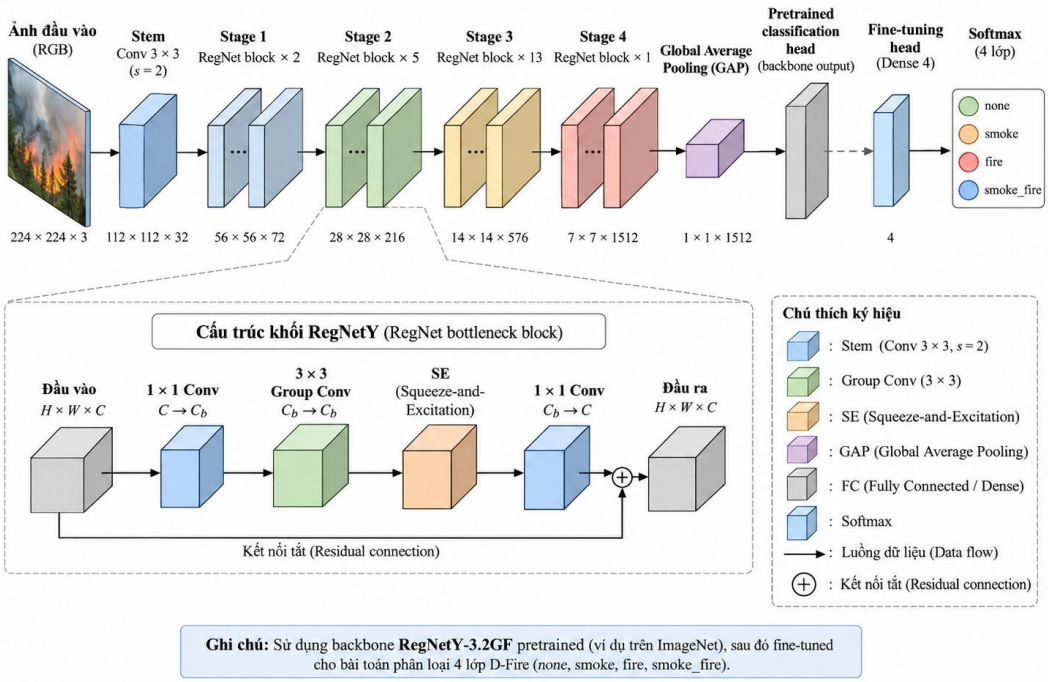
Hình 4 mô tả luồng kiến trúc từ stem qua bốn stage trích xuất đặc trưng đến Global Average Pooling và đầu phân loại. Đây là hình vẽ lại mang tính khái quát theo RegNet [2] và thiết lập trong transfer_common.py, không gán số khối lặp giả định cho từng stage. Việc tái sử dụng trọng số ImageNet-1K tạo điểm khởi tạo khác biệt căn bản so với CNN-K7 huấn luyện từ đầu.

Trong khối Squeeze-and-Excitation (SE), thông tin toàn cục trên mỗi kênh được rút gọn bằng GAP rồi truyền qua các phép biến đổi tuyến tính, hàm kích hoạt và sigmoid để tạo trọng số hiệu chỉnh theo kênh [6]. Dạng khái quát:

$$s = \text{sigmoid}(W_2\,\text{ReLU}(W_1\,\text{GAP}(U)))$$

$$\widetilde{U}_c = s_c\,U_c$$

Trong đó U là đặc trưng đầu vào, s_c là hệ số chú ý của kênh c, và Ũ là đặc trưng đã được điều chỉnh. Sơ đồ RegNetY trong Hình 4 biểu diễn các stage ở cấp chức năng, không gán số lượng bottleneck giả định cho từng stage.



Hình 2. Kiến trúc mô hình RegNetY-3.2GF dùng để so sánh.

Hình 4. Kiến trúc RegNetY-3.2GF tiền huấn luyện và đầu phân loại bốn lớp (theo RegNet [2] và mã nguồn dự án).

Quá trình huấn luyện gồm hai giai đoạn. Giai đoạn đầu đóng băng backbone và chỉ tối ưu lớp phân loại trong 3 epoch ở learning rate 0,001; giai đoạn sau mở toàn bộ backbone để fine-tune với learning rate 0,0001. Optimizer AdamW, weight decay

0,0001 và CrossEntropyLoss được sử dụng xuyên suốt; checkpoint tốt nhất được chọn theo validation loss. Mã nguồn đặt tối đa 30 epoch và early stopping patience=7; lần chạy thực tế kết thúc sau 13 epoch.

3.5. Tổ chức thực nghiệm và lưu kết quả

Hai nhánh CNN-K7 và RegNetY-3.2GF dùng cùng ảnh nhưng lưu kết quả độc lập. CNN-K7 lưu checkpoint, lịch sử và metric trong outputs/k7, hình ảnh trong figures/k7. RegNetY lưu các tệp tương ứng trong outputs/regnet_y_3_2gf và figures/regnet_y_3_2gf. Cách bố trí này giúp truy vết mỗi biểu đồ đến đúng lịch sử huấn luyện và tránh ghi đè checkpoint của kiến trúc khác.

Bảng 4 tổng hợp các siêu tham số và khác biệt thiết kế chính. Vì hai mô hình có mức tiền huấn luyện, số tham số và diễn biến stopping khác nhau, số epoch hoặc thời gian huấn luyện không nên được xem là đại lượng công bằng tuyệt đối nếu không tái lập trên cùng cấu hình phần cứng và giao thức benchmark.

Bảng 4. Đối chiếu thiết lập huấn luyện của hai mô hình theo mã nguồn.

Tiêu chí	CNN-K7	RegNetY-3.2GF
Nguồn trọng số	Khởi tạo mới	ImageNet-1K V2
Lớp phân loại	Linear 128→4	Linear 1512→4
Batch size	16	16
Ảnh đầu vào	224×224 RGB	224×224 RGB
Tối ưu	AdamW	AdamW
Learning rate	0,001 (toàn mạng)	0,001 head; 0,0001 fine-tune
Weight decay	0,0001	0,0001
Hàm mất mát	CrossEntropyLoss	CrossEntropyLoss
Epoch tối đa / patience	30 / 7	30 / 7
Epoch thực tế	30	13
Chọn checkpoint	Validation loss nhỏ nhất	Validation loss nhỏ nhất

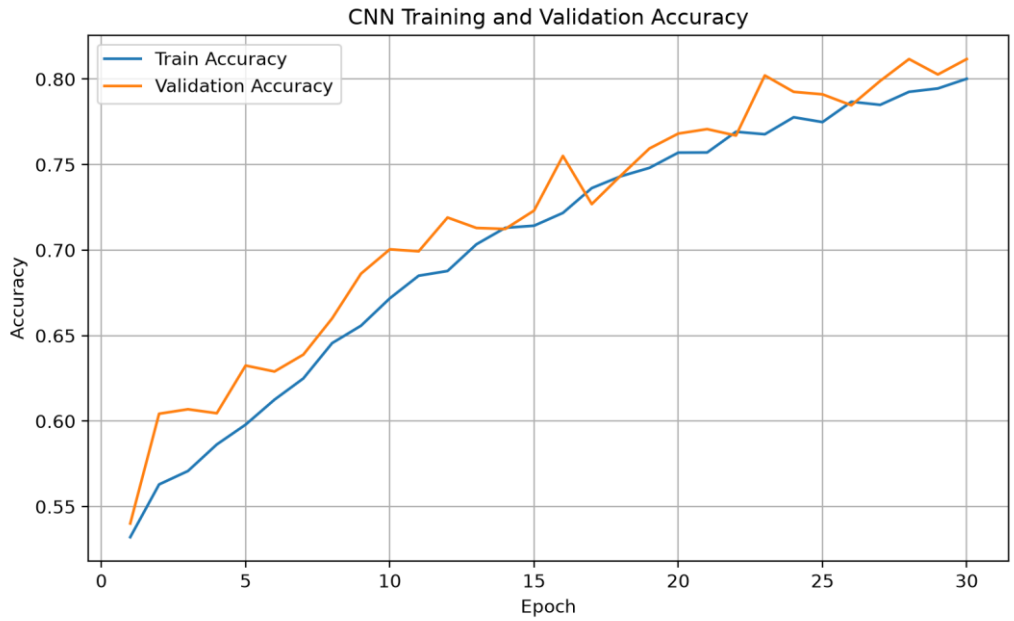
4. Thực nghiệm và kết quả

4.1. Kết quả quá trình huấn luyện CNN-K7

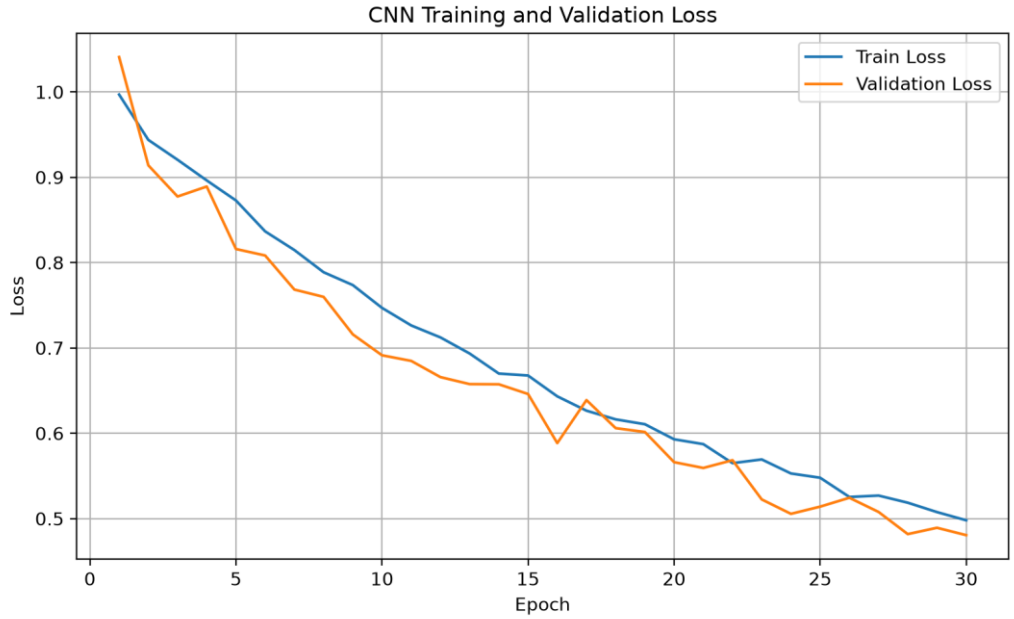
Lịch sử CNN-K7 ghi đủ 30 epoch. Ở epoch đầu, Train Accuracy đạt 53,21% và Validation Accuracy 54,01%; tới epoch 30, các giá trị này lần lượt là 80,02% và 81,17%. Validation loss giảm từ 1,0408 còn 0,4805; checkpoint theo validation loss tốt nhất thuộc epoch 30. Tổng thời gian cộng từ elapsed_seconds trong history.csv vào khoảng 297,7 phút trong phiên chạy được lưu.

Hình 5 thể hiện hai đường accuracy và Hình 6 thể hiện hai đường loss. Đường validation biến động theo từng epoch nhưng nhìn chung có xu hướng tiến triển theo

hướng có lợi; ở cuối tiến trình chưa thấy hiện tượng validation loss tăng liên tục trong khi train loss giảm. Kết quả huấn luyện này chứng tỏ mô hình học được quy luật trên tập D-Fire, nhưng chưa xác nhận năng lực phân loại đủ mạnh ở từng lớp riêng biệt.



Hình 5. Đường cong Train Accuracy và Validation Accuracy của CNN-K7 qua 30 epoch (kết quả thực nghiệm).



Hình 6. Đường cong Train Loss và Validation Loss của CNN-K7 qua 30 epoch (kết quả thực nghiệm).

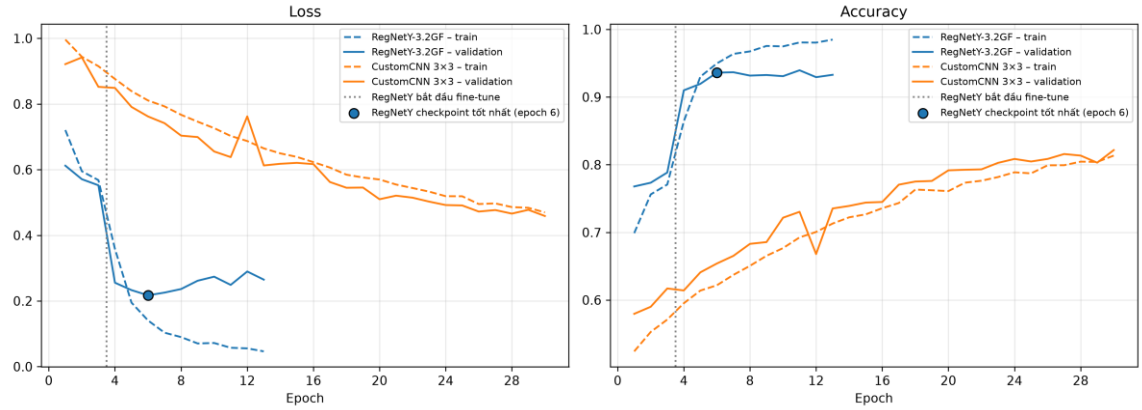
4.2. Kết quả quá trình huấn luyện RegNetY-3.2GF

Đường lịch sử RegNetY ghi 3 epoch huấn luyện phần đầu phân loại và 10 epoch fine-tune toàn mạng. Chỉ số validation tăng nhanh sau khi mở backbone: từ 78,85% ở epoch 3 lên 90,99% ở epoch 4. Validation loss thấp nhất 0,2172 thuộc epoch 6, với Validation Accuracy 93,61%. Dù Validation Accuracy có lúc cao hơn ở các epoch

muộn, checkpoint được chọn theo validation loss nên trọng số epoch 6 được dùng để đánh giá tập test.

Ở RegNetY, huấn luyện kéo dài 13 epoch; checkpoint theo validation loss thấp nhất được ghi nhận ở epoch 6 (val loss xấp xỉ 0,2172). Mặc dù một số epoch về sau đạt validation accuracy cao hơn, checkpoint được đánh giá Test vẫn được chọn nhất quán theo tiêu chí validation loss, không chọn theo dữ liệu Test.

Hình 7 trình bày biểu đồ learning curves được tạo trong nhánh RegNetY. Ở các epoch sau checkpoint tốt nhất, train accuracy tiếp tục tăng đến khoảng 98,49% trong khi validation loss có xu hướng tăng trở lại và validation accuracy dao động; điều này gợi ý dấu hiệu quá khớp ở giai đoạn tinh chỉnh muộn, và cho thấy ích lợi của cơ chế chọn checkpoint theo validation loss thay vì sử dụng epoch cuối.



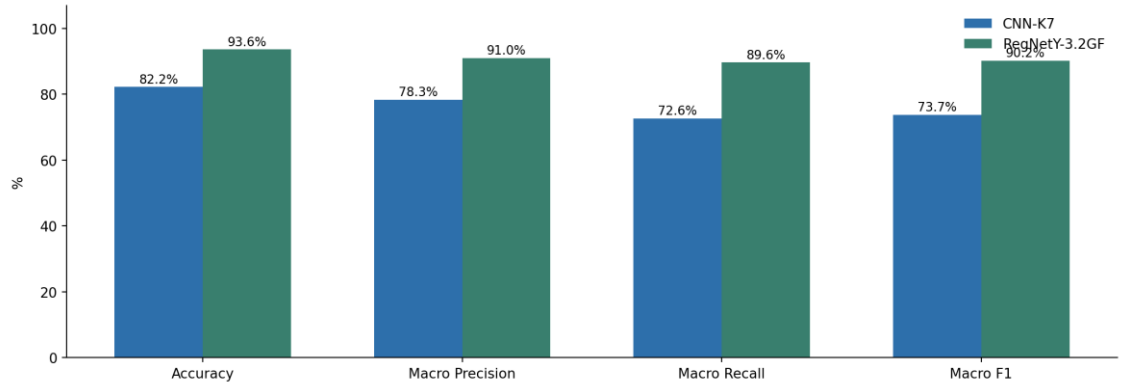
Hình 7. Diễn biến huấn luyện và xác thực của RegNetY-3.2GF, gồm giai đoạn head và fine-tune (kết quả dự án).

4.3. Kết quả tổng hợp trên tập Test

Các phép đánh giá sử dụng cùng 4.306 ảnh Test với đúng thứ tự nhãn. Bảng 5 và Hình 8 trình bày độ chính xác và các chỉ số trung bình Macro. Theo tệp test_metrics.json, CNN-K7 đạt Accuracy 82,23%, Macro F1 73,75%; trong khi RegNetY-3.2GF đạt Accuracy 93,59%, Macro F1 90,22%. Chênh lệch Accuracy là 11,36 điểm phần trăm và chênh lệch Macro F1 là 16,47 điểm phần trăm, cho thấy ưu thế của RegNet rõ hơn khi chú ý tới sự cân bằng giữa các lớp.

Bảng 5. So sánh hiệu quả hai kiến trúc trên 4.306 ảnh Test.

Chỉ số trên Test	CNN-K7	RegNetY-3.2GF	Chênh lệch (điểm %)
Accuracy	82,23%	93,59%	+11,36
Macro Precision	78,34%	90,99%	+12,64
Macro Recall	72,62%	89,62%	+17,00
Macro F1	73,75%	90,22%	+16,47

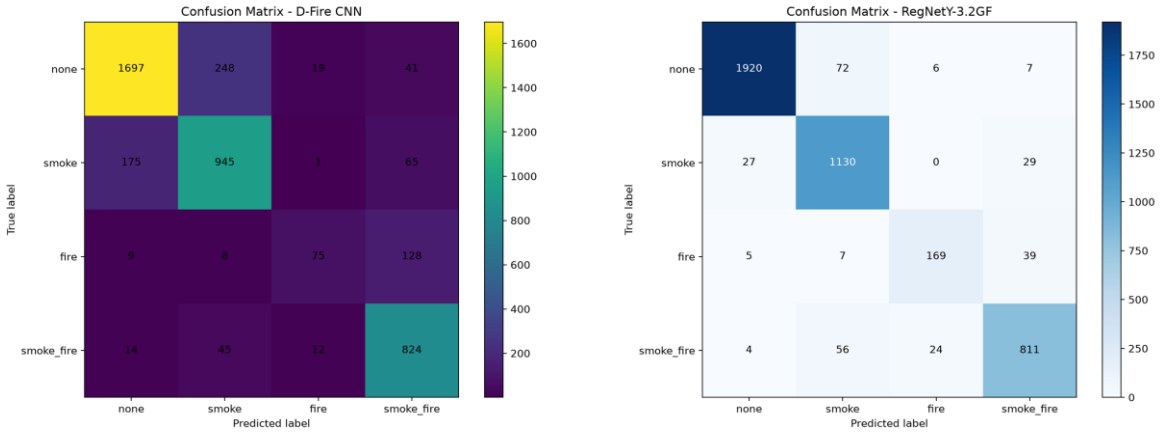


Hình 8. So sánh Accuracy và các chỉ số Macro của CNN-K7 với RegNetY-3.2GF (số liệu thực nghiệm).

Trong bối cảnh giám sát an toàn, sự vượt trội về Macro Recall và Macro F1 đặc biệt đáng chú ý vì tập kiểm tra mất cân bằng; RegNet không chỉ cải thiện dự đoán ở lớp chiếm đa số. Dù vậy, Accuracy 93,59% vẫn chưa đủ để khẳng định một hệ thống cảnh báo hoàn chỉnh phù hợp triển khai thực địa: cần tiếp tục kiểm nghiệm theo điều kiện ánh sáng, nguồn camera và chi phí báo động sai.

4.4. Phân tích ma trận nhầm lẫn và lớp fire

Hình 9 trình bày hai ma trận nhầm lẫn, với hàng là nhãn thực tế và cột là nhãn dự đoán. CNN-K7 nhận đúng 3.541/4.306 ảnh; RegNetY nhận đúng 4.030/4.306 ảnh. Trên riêng 220 ảnh fire, CNN-K7 nhận đúng 75 ảnh và nhầm 128 ảnh thành smoke_fire; RegNetY nhận đúng 169 ảnh và nhầm 39 ảnh thành smoke_fire. Đây là nguyên nhân trọng yếu dẫn tới khác biệt về Recall của lớp fire giữa hai mô hình.

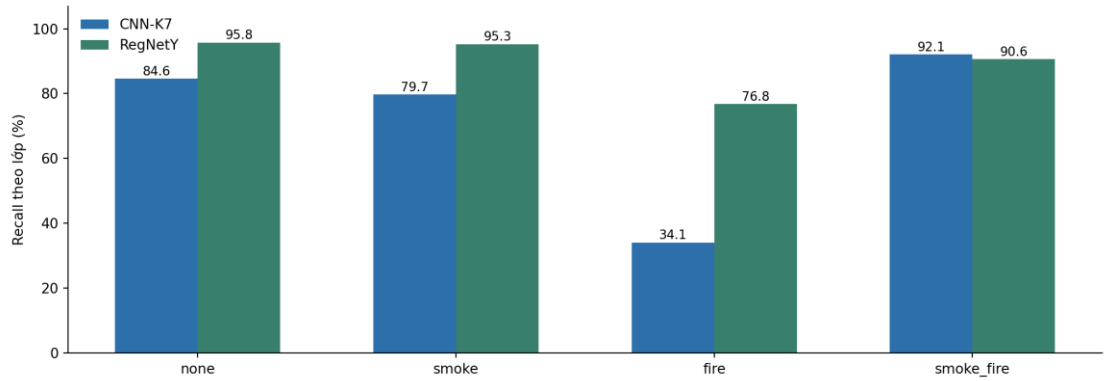


Hình 9. Ma trận nhầm lẫn trên cùng tập Test: CNN-K7 (trái) và RegNetY-3.2GF (phải).

Bảng 6 tổng hợp Recall theo từng lớp. Lớp smoke_fire là trường hợp duy nhất CNN-K7 có Recall cao hơn RegNetY trong lần thử nghiệm này, nhưng chênh lệch nhỏ; ở ba lớp còn lại RegNetY hoạt động tốt hơn, đặc biệt ở fire. Bảng và Hình 10 cho thấy lý do Macro Recall chênh lệch mạnh dù lớp none chiếm phần lớn số ảnh.

Bảng 6. Recall theo lớp, tính từ ma trận nhầm lẫn của hai mô hình.

Lớp	Số ảnh Test	CNN-K7 Recall	RegNetY Recall
none	2.005	84,64%	95,76%
smoke	1.186	79,68%	95,28%
fire	220	34,09%	76,82%
smoke_fire	895	92,07%	90,61%



Ngoài nhiệm vụ phân loại bốn lớp, có thể đánh giá bổ sung chỉ tiêu cảnh báo "có lửa" bằng cách xem hai nhãn fire và smoke_fire là dương tính, hai nhãn none và smoke là âm tính. Khi đó Recall của nhóm có lửa là:

$$\text{Recall}_{\text{has_fire}} = \frac{TP_{\text{has_fire}}}{TP_{\text{has_fire}} + FN_{\text{has_fire}}}$$

Từ ma trận nhầm lẫn của CNN-K7: trong 1.115 ảnh thực sự có lửa, 1.039 ảnh được dự đoán vào nhóm có lửa, cho Recall 93,18%; Precision của nhóm này là 1.039/1.165 = 89,18%. Với RegNetY, chỉ số đã lưu trong kết quả đánh giá tương ứng Recall 93,54% và Precision 96,13%. Các giá trị nhị phân không thay thế kết quả bốn lớp, mà giúp xác định khác biệt giữa lỗi phân biệt trạng thái và lỗi bỏ sót cảnh báo.

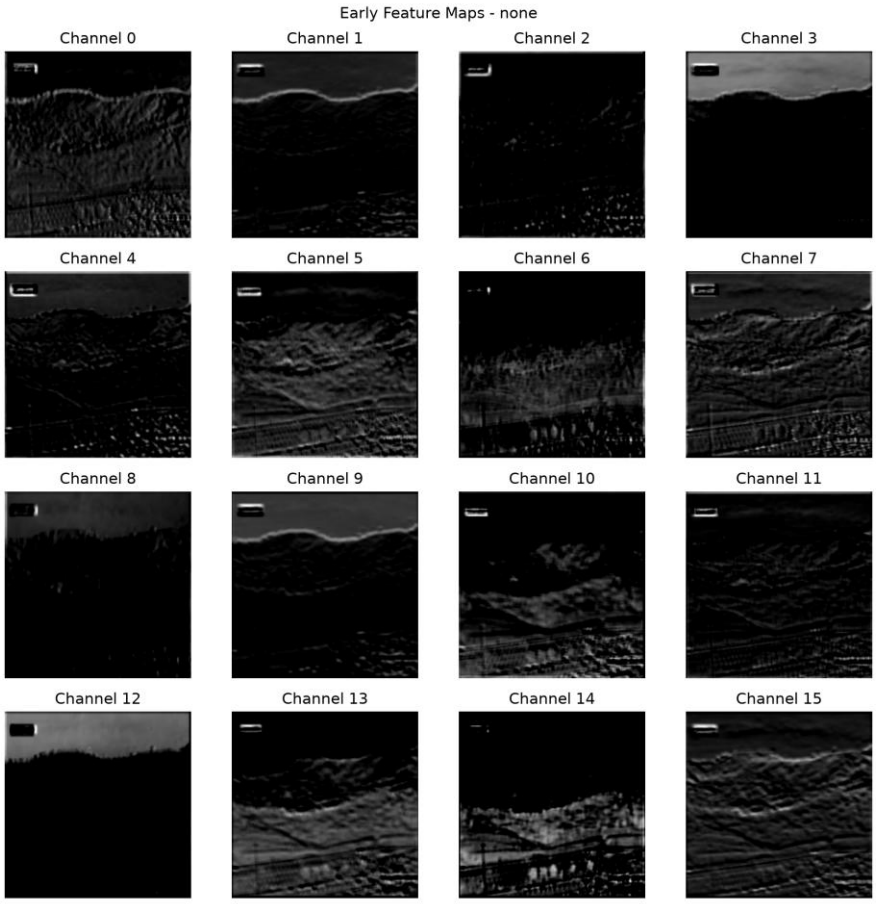
Hình 10. Recall theo từng lớp: CNN-K7 và RegNetY-3.2GF (tính lại từ ma trận nhầm lẫn).

Nếu bài toán được chuyển từ phân loại bốn lớp sang cảnh báo nhị phân "có lửa", có thể gộp dự đoán fire và smoke_fire làm nhãn dương. Tính từ các ma trận nhầm lẫn, CNN-K7 có Precision khoảng 89,18% và Recall 93,18% với nhãn gộp; RegNetY có Precision 96,13% và Recall 93,54%. Kết quả này cho thấy tỷ lệ bỏ sót sự hiện diện của lửa giữa hai mô hình gần nhau hơn so với Recall của riêng nhãn fire, nhưng RegNetY vẫn tạo ít dự đoán báo lửa sai hơn.

Các chỉ số gộp chỉ là phép tính lại trên nhãn của tập D-Fire; chúng không phải đánh giá độ tin cậy của một hệ thống cảnh báo thực tế và không phản ánh trực tiếp những điều kiện ghi hình mới ngoài tập dữ liệu.

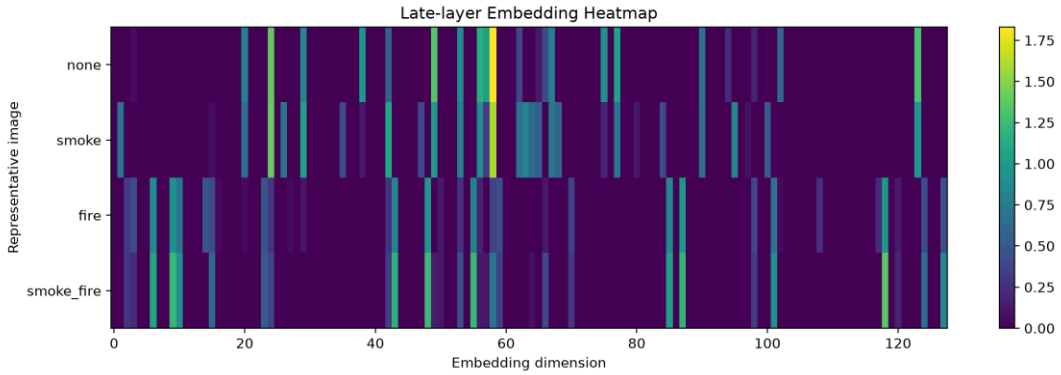
4.5. Trực quan hóa feature maps, embedding và trường hợp sai

Hình 11 cho thấy 16 trong số 32 feature map sau ReLU của Block1, được trích từ một ảnh none đại diện. Các kênh nhấn mạnh vùng biên và cấu trúc với cường độ khác nhau; bản đồ kích hoạt sáng hơn chỉ có nghĩa bộ lọc phản ứng mạnh hơn tại vị trí tương ứng. Hình này không phải bằng chứng độc lập rằng CNN đã nhận biết đúng khói hoặc lửa.



Hình 11. Mười sáu feature map đầu tiên sau Block1 của CNN-K7, tạo từ một ảnh đại diện thuộc lớp none.

Hình 12 trình bày embedding 128 chiều của bốn ảnh đại diện, mỗi lớp một ảnh. Những mẫu kích hoạt khác nhau gợi ý mạng đã học biểu diễn không hoàn toàn trùng nhau; tuy nhiên, chỉ bốn mẫu không đủ để kết luận đặc trưng của các lớp đã phân tách tốt. Những nhận định định lượng phải dựa trên toàn tập Test, đặc biệt ma trận nhầm lẫn và F1.



Hình 12. Heatmap embedding 128 chiều cho bốn ảnh đại diện từ CNN-K7.

Hình 13 minh họa 12 trường hợp ảnh có nhãn fire nhưng CNN-K7 dự đoán thành smoke_fire. Các ảnh minh họa chủ yếu có nền đêm tối, nguồn sáng lửa nhỏ và điều kiện quan sát không thuận lợi. Sự giống nhau giữa một số khung hình cho thấy cần kiểm tra kỹ tính đa dạng ngữ cảnh và khả năng tương quan giữa các ảnh; tuy nhiên chỉ riêng quan sát hình không đủ để khẳng định nguyên nhân sai là do ánh sáng hay do gán nhãn.



Hình 13. Mười hai trường hợp fire bị dự đoán nhầm thành smoke_fire bởi CNN-K7 (mẫu lỗi được chọn để trực quan hóa).

Lưu ý số 12 ở Hình 13 là số ảnh ví dụ mà chương trình phân tích lựa chọn; tổng số ảnh fire bị nhầm sang smoke_fire trên Test là 128 theo ma trận nhầm lẫn. Việc phân biệt mẫu minh họa với toàn bộ số lỗi giúp tránh diễn giải quá mức hiệu quả của mô hình.

4.6. Ưu, nhược điểm và tính phù hợp triển khai

Bảng 7 tổng hợp ưu điểm và hạn chế của hai mô hình. CNN-K7 có 426.628 tham số, RegNetY-3.2GF có 17.929.390 tham số sau khi thay classifier, tức lớn hơn khoảng 42 lần. Theo các tệp history.csv, thời gian huấn luyện cộng dồn khoảng 297,7 phút đối với CNN-K7 và 52,3 phút đối với RegNetY trên những lần chạy được ghi lại. Thời gian này chịu ảnh hưởng bởi phần cứng, việc nạp dữ liệu và chiến lược huấn luyện; không được hiểu là so sánh tốc độ mạng trên cùng một thí nghiệm chuẩn hóa.

Bảng 7. Ưu điểm và hạn chế của hai mô hình theo kết quả và cấu trúc thực nghiệm.

Khía cạnh	CNN-K7	RegNetY-3.2GF
Kiến trúc	4 khối Conv; Conv1 7×7; GAP + MLP	Backbone nhiều stage, pretrained; head 4 lớp
Tham số sau thiết kế	426.628 (~0,427 M)	17.929.390 (~17,929 M)
Độ chính xác Test	82,23%	93,59%
Recall lớp fire	34,09%	76,82%
Ưu điểm chính	Nhỏ gọn, dễ mô tả, đáp ứng điều kiện Conv1 7×7	Phân loại cân bằng hơn; khởi tạo pretrained
Hạn chế chính	Nhầm fire↔smoke_fire; khả năng biểu diễn còn yếu	Chi phí bộ nhớ lớn; tiềm ẩn overfitting khi fine-tune

Xét mục tiêu phân loại bốn trạng thái trong tập dữ liệu hiện có, RegNetY-3.2GF là lựa chọn có chất lượng tốt hơn. Đối với mục tiêu nghiên cứu nguyên lý CNN, CNN-K7 giữ vai trò đối chứng nhỏ gọn và minh họa trực tiếp tác động của thiết kế lớp tích chập đầu tiên. Để lựa chọn mô hình vận hành trên thiết bị thực, cần bổ sung benchmark cùng phần cứng về độ trễ, FPS, mức sử dụng VRAM, công suất và khả năng duy trì hoạt động liên tục.

5. Kết luận

5.1. Kết quả đạt được

Thực nghiệm đã hoàn thành hai hướng phân loại ảnh D-Fire bốn lớp: CNN tự xây dựng có Conv1 kernel 7×7 và RegNetY-3.2GF fine-tune từ ImageNet-1K V2. Cùng 21.527 ảnh được phân chia thành train, validation và test với các manifest thống nhất. CNN-K7 hội tụ sau 30 epoch và đạt 82,23% Accuracy, 73,75% Macro F1; RegNetY dừng sau 13 epoch, checkpoint được chọn ở epoch 6 và đạt 93,59% Accuracy, 90,22% Macro F1.

RegNetY-3.2GF cho kết quả phân loại bốn lớp tốt hơn, đặc biệt ở lớp fire. Trái lại, CNN-K7 có số tham số nhỏ hơn đáng kể và kiến trúc rõ ràng, phù hợp làm mô hình nền để đánh giá các lựa chọn thiết kế. Việc chỉ sử dụng Accuracy sẽ chưa phản ánh đúng chất lượng ở lớp ít mẫu; các chỉ số Macro và phân tích lỗi là cần thiết để rút ra kết luận nhất quán.

5.2. Giới hạn và hướng phát triển

Giới hạn chính của nghiên cứu gồm: số ảnh fire ít hơn nhiều so với lớp none; khó khăn trong phân biệt fire với smoke_fire; các ảnh lỗi có bối cảnh ban đêm tương tự; chưa đo chính thức khả năng suy luận trên cùng thiết bị triển khai; chưa đánh giá trên camera hoặc tập ảnh thu thập độc lập. Dữ liệu chỉ hỗ trợ kết luận về phân loại ảnh tĩnh, không chứng minh đáp ứng độ trễ hay độ tin cậy của một bộ báo cháy thực tế.

Các hướng tiếp tục gồm cân bằng lại phân bố lớp bằng phương pháp lấy mẫu có kiểm soát hoặc trọng số lớp trong hàm mất mát; bổ sung ảnh fire nhỏ ở các điều kiện sáng khác nhau; thử nghiệm kỹ thuật phát hiện vùng đối tượng trước phân loại; đo thêm chỉ số cảnh báo nhị phân; và benchmarking mô hình trên cùng thiết bị mục tiêu. Những thử nghiệm tiếp theo cần duy trì tập Test độc lập, không dùng Test để lựa chọn siêu tham số, đồng thời báo cáo trung bình và độ lệch chuẩn qua nhiều seed.

TÀI LIỆU THAM KHẢO

[1] P. V. A. B. de Venâncio, A. C. Lisboa và A. V. Barbosa, “An automatic fire detection system based on deep convolutional neural networks for low-power, resource-constrained devices,” Neural Computing and Applications, 2022.

[2] I. Radosavovic, R. P. Kosaraju, R. Girshick, K. He và P. Dollár, “Designing Network Design Spaces,” Proceedings of IEEE/CVF CVPR, tr. 10428–10436, 2020.

[3] A. Krizhevsky, I. Sutskever và G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” Advances in Neural Information Processing Systems, tập 25, 2012.

[4] S. Ioffe và C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” ICML, 2015.

[5] K. He, X. Zhang, S. Ren và J. Sun, “Deep Residual Learning for Image Recognition,” CVPR, 2016.

[6] J. Hu, L. Shen và G. Sun, “Squeeze-and-Excitation Networks,” CVPR, 2018.

[7] I. Loshchilov và F. Hutter, “Decoupled Weight Decay Regularization,” International Conference on Learning Representations, 2019.

[8] PyTorch TorchVision, “regnet_y_3_2gf,” https://docs.pytorch.org/vision/main/models/generated/torchvision.models.regnet_y_3_2gf.html (truy cập tháng 10/2026).

[9] Gaia Solutions on Demand, “D-Fire: an image dataset for fire and smoke detection,” https://github.com/gaia-solutions-on-demand/DFireDataset (truy cập tháng 10/2026).

[10] J. Deng và cộng sự, “ImageNet: A Large-Scale Hierarchical Image Database,” IEEE Conference on Computer Vision and Pattern Recognition, 2009.

[11] A. Paszke và cộng sự, “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” NeurIPS, 2019.

PHỤ LỤC A. TRÍCH ĐOẠN MÃ NGUỒN TIÊU BIỂU

Phụ lục A trích các đoạn mã then chốt từ hai dự án, tập trung vào kiến trúc, tiền xử lý, quá trình tối ưu và đánh giá trên tập Test. Các đoạn mã dùng trong phụ lục mang tính minh họa chức năng; các file Python đầy đủ được lưu trong bộ mã nguồn của từng mô hình.

A.1. Lớp Conv1 kernel 7×7 của CNN-K7

Cửa sổ 7×7, padding 3, 32 kênh, BatchNorm, ReLU và MaxPool (Nguồn mã: CNN_K7/src/model_k7.py).

```
self.block1 = nn.Sequential(
    nn.Conv2d(
        in_channels=3,
        out_channels=32,
        kernel_size=7,
        padding=3
    ),
    nn.BatchNorm2d(32),
    nn.ReLU(inplace=False),
    nn.MaxPool2d(kernel_size=2)
)
```

A.2. Các khối tích chập kế tiếp và tầng phân loại của CNN-K7

Ba khối tiếp theo dùng kernel 3×3, GAP và đầu phân loại bốn lớp (Nguồn mã: CNN_K7/src/model_k7.py).

```
self.block4 = self._block(
    in_channels=128,
    out_channels=256,
    pool=False
)

self.gap = nn.AdaptiveAvgPool2d(1)

self.embedding = nn.Sequential(
    nn.Linear(256, 128),
    nn.ReLU(inplace=False),
    nn.Dropout(0.30),
)

self.classifier = nn.Linear(
    128,
    num_classes
```

A.3. Luồng dữ liệu qua mô hình CNN-K7

Thứ tự trích xuất đặc trưng, GAP, embedding và logits (Nguồn mã: CNN_K7/src/model_k7.py).

```
def forward(self, x):
    x = self.block1(x)
    x = self.block2(x)
    x = self.block3(x)
    x = self.block4(x)

    x = self.gap(x)
    x = x.flatten(1)

    x = self.embedding(x)

    logits = self.classifier(x)
```

```
    return logits
```

A.4. Một bước cập nhật gradient của CNN-K7

CrossEntropyLoss, tính gradient và cập nhật bằng AdamW (Nguồn mã: CNN_K7/src/train_k7.py).

```
optimizer.zero_grad(
    set_to_none=True
)

logits = model(images)

loss = criterion(
    logits,
    labels
)

loss.backward()

optimizer.step()
```

A.5. Lưu checkpoint tốt nhất theo validation loss của CNN-K7

Lưu epoch, state_dict của mô hình và trạng thái optimizer khi kết quả xác thực cải thiện (Nguồn mã: CNN_K7/src/train_k7.py).

```
def save_best_checkpoint(
    model,
    optimizer,
    epoch,
    train_loss,
    train_accuracy,
    val_loss,
    val_accuracy,
):
    OUTPUT_DIR.mkdir(
    checkpoint = {
        "epoch": epoch,
        "model_state_dict": model.state_dict(),
        "optimizer_state_dict": optimizer.state_dict(),
        "val_loss": val_loss,
        "val_accuracy": val_accuracy,
    }
    torch.save(checkpoint, BEST_MODEL_PATH)
```

A.6. Tính các chỉ số đánh giá CNN-K7 trên Test

Thu thập dự đoán và tổng hợp Accuracy, Macro Precision, Macro Recall, Macro F1 (Nguồn mã: CNN_K7/src/evaluate_k7.py).

```
accuracy = accuracy_score(y_true, y_pred)
macro_precision = precision_score(
    y_true, y_pred, average="macro", zero_division=0)
macro_recall = recall_score(
    y_true, y_pred, average="macro", zero_division=0)
macro_f1 = f1_score(
    y_true, y_pred, average="macro", zero_division=0)
matrix = confusion_matrix(y_true, y_pred)
```

A.7. Khởi tạo RegNetY-3.2GF và thay lớp phân loại

Tải pretrained ImageNet-1K V2 và thay lớp cuối thành bốn logits (Nguồn mã: RegNetY/src/transfer_common.py).

```
def _regnet_y_3_2gf(pretrained):
    # IMAGENET1K_V2: 82.6% top-1 ImageNet
    weights = models.RegNet_Y_3_2GF_Weights.IMAGENET1K_V2 if pretrained else None
    model = models.regnet_y_3_2gf(weights=weights)
    model.fc = nn.Linear(model.fc.in_features, NUM_CLASSES)   # 1512 -> 4
    return model
```

A.8. Đóng băng và mở băng backbone ở RegNetY

Điều khiển requires_grad cho các tham số ngoài lớp fc (Nguồn mã: RegNetY/src/transfer_common.py).

```
def set_backbone_trainable(model, trainable, head_name):
    """Đóng băng / mở băng mọi lớp, trừ lớp phân loại cuối (head_name)."""
    for name, parameter in model.named_parameters():
        if not name.startswith(head_name + "."):
            parameter.requires_grad = trainable
```

A.9. Huấn luyện hai giai đoạn RegNetY-3.2GF

Ba epoch đầu train head với learning rate 10^{-3}; tiếp theo fine-tune toàn mạng với 10^{-4} (Nguồn mã: RegNetY/src/train_transfer.py).

```
HEAD_EPOCHS = 3
MAX_EPOCHS = 30
EARLY_STOPPING_PATIENCE = 7

LR_HEAD = 1e-3
LR_FINETUNE = 1e-4
WEIGHT_DECAY = 1e-4

set_backbone_trainable(model, False, spec.head_name)
phase, current_lr = "head", LR_HEAD
optimizer = make_optimizer(model, current_lr)

if epoch == HEAD_EPOCHS + 1:
    set_backbone_trainable(model, True, spec.head_name)
    phase, current_lr = "finetune", LR_FINETUNE
    optimizer = make_optimizer(model, current_lr)
```

A.10. Đánh giá và lưu kết quả RegNetY trên tập Test

Nạp checkpoint tốt nhất, dự đoán tập Test, lưu JSON metrics và Confusion Matrix (Nguồn mã: RegNetY/src/evaluate_transfer.py).

```
model = spec.build(pretrained=False).to(device)
checkpoint = torch.load(spec.best_model_path, map_location=device)
model.load_state_dict(checkpoint["model_state_dict"])

print(f"Model: {spec.display_name}")
print(f"Loaded best checkpoint từ epoch {checkpoint['epoch']} "
      f"({checkpoint['phase']})")
print(f"Best val loss    : {checkpoint['val_loss']:.4f}")
print(f"Best val accuracy: {checkpoint['val_accuracy']:.4f}")
print("\nĐang dự đoán tập test (mất vài phút, không in gì trong lúc chạy)...")

y_true, y_pred = evaluate_test(model, test_loader, device)
print_metrics(y_true, y_pred)
with open(spec.metrics_path, "w", encoding="utf-8") as f:
    json.dump(compute_metrics(y_true, y_pred), f, indent=4)
```

```
save_confusion_matrix(spec, y_true, y_pred)
```

PHỤ LỤC B. THÔNG TIN THÀNH VIÊN VÀ CAM KẾT NHÓM

Bảng 8 là bảng phân công nhiệm vụ và minh chứng đóng góp của ba thành viên. Họ tên, mã số sinh viên, thông tin thực hiện, minh chứng và chữ ký được điền theo phân công và đóng góp thực tế của nhóm.

Bảng 8. Bảng phân công và minh chứng đóng góp của thành viên nhóm (bổ sung thông tin thực tế).

STT	Họ và tên / MSSV	Vai trò / nhiệm vụ cụ thể	Minh chứng đóng góp	Ký tên
1	Nguyễn Tấn Vỹ	CNN-K7 (Conv1 7×7) và phân tích lỗi: sửa kiến trúc theo yêu cầu kernel 7×7, huấn luyện, đánh giá, so sánh với bản 3×3, trực quan hóa feature map, embedding và các mẫu dự đoán sai. Viết: Mục 3.3, 4.1, 4.4 – 4.6, 5, phụ lục A. Phụ trách 33%.	model_k7.py, train_k7.py, evaluate_k7.py; outputs/k7/, figures/k7/; Hình 3, 5, 6, 9 – 13	.....
2	Nguyễn Trường Thọ	Dữ liệu và CNN gốc Conv1 3×3: kiểm tra dữ liệu D-Fire, tạo manifest train/val/test, xây dựng data.py (tiền xử lý, tăng cường dữ liệu), cài đặt và huấn luyện CNN 3×3 làm mô hình nền. Viết: Mục 2.1, 2.2, 2.4, 2.5, 3.1, 3.2. Phụ trách 33%.	data_audit.py, build_manifest.py, data.py, model.py, train.py, evaluate.py; thư mục manifests/; Bảng 2, Hình 2	.....

STT	Họ và tên / MSSV	Vai trò / nhiệm vụ cụ thể	Minh chứng đóng góp	Ký tên
3	Nguyễn Đức Trúc	Mô hình RegNetY-3.2GF (học chuyển giao): chọn mô hình, cài đặt pipeline train 2 giai đoạn (đóng băng backbone, rồi fine-tune), huấn luyện, đánh giá trên tập Test, phân tích overfitting. Viết: Mục 1, 2.3, 3.4, 4.2, 4.3. Phụ trách 33%.	transfer_common.py, train_transfer.py, evaluate_transfer.py; outputs/regnet_y_3_2gf/ (checkpoint, history.csv, test_metrics.json); Hình 4, 7, 8	.....

Cam kết nhóm

Các thành viên cam kết phân công đúng thực tế, không ký tên hình thức; hiểu và có thể giải thích phần việc phụ trách; chấp nhận điều chỉnh điểm cá nhân theo quy chế học phần. Việc ký tên được thực hiện sau khi kiểm tra minh chứng đóng góp.

Đại diện nhóm ký tên: Ngày: 29/10/2026